I2ML Report

Chloé Djoussa Yan Zhen Ho
Julius Schlensok (Kaggle team 100)

December 2024

1 Introduction

This term project explores the use of machine learning methods to predict saturation vapour pressure (p_{Sat}) for atmospheric molecules, leveraging the GeckoQ dataset. The GeckoQ dataset contains the atomic structures and thermodynamic properties of 31,637 molecules formed from the oxidation of pinene, toluene, and decane.

Saturation vapour pressure is a critical property in atmospheric science as it indicates a molecule's ability to condense into aerosol particles. Molecules with low p_{Sat} are classified as low-volatile organic compounds (LVOCs), which play a pivotal role in new particle formation (NPF) and aerosol particle growth. These processes significantly impact air quality and climate, influencing phenomena such as global cooling and air pollution.

Computationally, p_{Sat} is typically derived from quantum chemistry simulations—a resource-intensive approach that limits scalability. As the atmospheric system involves a vast array of molecules, the development of efficient regression models offers a faster, cost-effective alternative for estimating this property and enabling larger-scale atmospheric modeling.

Our challenge in this project was to design a machine learning model capable of accurately predicting p_{Sat} using the interpretable features provided in the GeckoQ dataset. This work aligns with the broader goals of improving our understanding of NPF processes and advancing predictive tools for atmospheric science.

2 Dataset

2.1 Data exploration

2.1.1 General observations

The dataset has been split into a training set of 26,237 samples and a test set containing 5,000. Each sample represents an individual molecule. In addition to the target variable, logarithmic saturation vapour pressure `log_pSat_Pa`, the dataset contains an ID field and 25 explainable features.

Looking at the datatypes of the features, most are integers, with only logarithmic saturation vapour pressure and molecular weight containing floating-point values and one single categorical feature `parentspecies` represented by a string indicating one or multiple parent species (out of the three options *apin*, *toluene* and *decane*). This last column is also the only one containing missing values, of which 210 appear in the training dataset and a further 33 in the test set. While parent species is semantically the only categorical feature, a total of 22 columns contain fewer than 10 unique values, the only exceptions being logarithmic saturation vapour pressure, molecular weight, number of atoms, number of oxygen atoms, number of stable conformers, and number of stable conformers used for thermodynamic property computation.

For further analysis, the categorical parent species feature was split into three binary features, *toluene*, *apin* and *decane*, each of them indicating whether a molecule had the respective compound as a predecessor (since descent can be ambiguous). Because the test set contains 33 missing parent species annotations, missing parent species annotations in the training set were encoded as negative (zero value) for all three new features instead of dropping the samples. The molecule index *ID* was also removed since it contained no relevant information.

2.1.2 Data distribution

Logarithmic saturation vapour pressure ranges from -18.82 to 8.39. It looks normally distributed with mean -5.52 and standard deviation 3.12 (Figure 9).

Of the variables with more than 10 unique values, none appear normally distributed. Molecular weight, number of atoms and number of oxygen atoms have a heavy left tail; number of stable conformers is extremely skewed towards small values, and number of conformers used for thermodynamic property computa-

tion has more than 10,000 samples with its mode value of 40, possibly because it was set as a computational limit in thermodynamic property calculation. Virtually all of the features with fewer than 10 unique values are extremely unequally distributed, the only exceptions being number of nitrogen atoms and number of hydrogen bond donors (Figure 9).

2.1.3 Correlation between features

There is significant covariation amongst the groups of structural features (number of total/carbon/hydrogen/oxygen atoms, molecular weight, number of hydrogen bond donors, number of stable conformers, number of conformers used for thermodynamic property calculation), and parent species (apine/toluene/decane), both within the groups and, to a lesser extent, with other features (Figure 1). Other features show low covariation with each other.

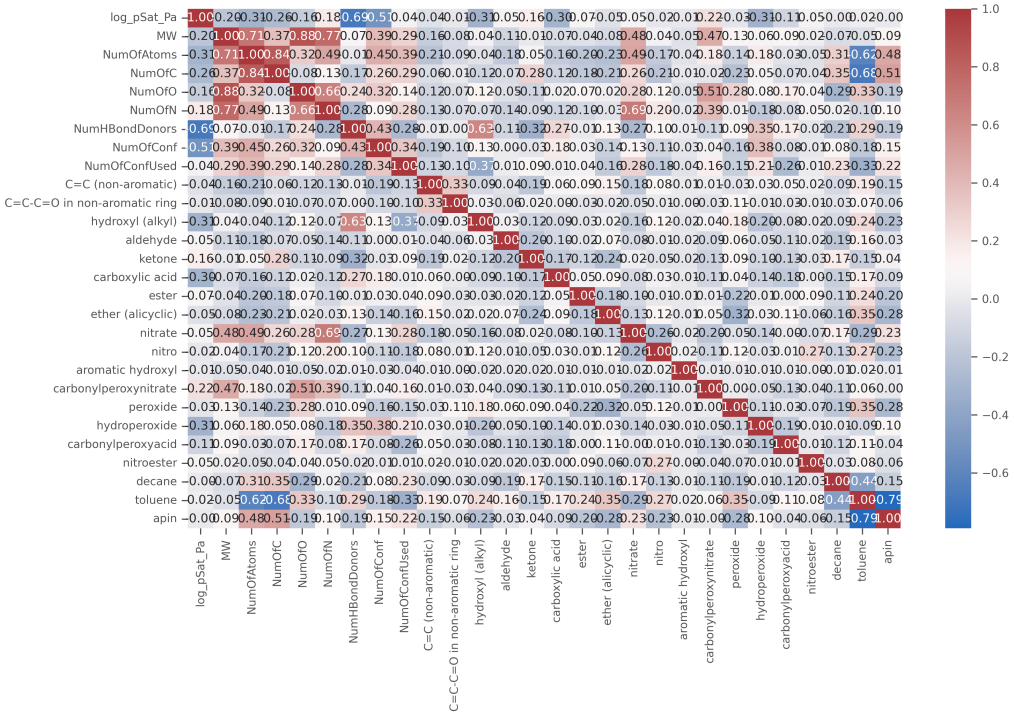


Figure 1: Correlation matrix of variables in training dataset. Correlation method is Pearson's correlation coefficient.

This covariation can pose a challenge for machine learning models, as highly

correlated features may provide redundant information, potentially complicating the training process and leading to overfitting.

The closest correlation with the target variable can be observed for `NumHBondDonors` (-0.69) and `NumOfConf` (-0.51) (**??**). 10 features show an absolute correlation of less than 5 % with saturation vapour pressure (Table 1).

Feature	**Pearson** r
NumHBondDonors	-0.689
NumOfConf	-0.514
hydroperoxide	-0.314
hydroxyl (alkyl)	-0.310
NumOfAtoms	-0.307
carboxylic acid	-0.304
NumOfC	-0.263
MW	-0.200
NumOfO	-0.155
carbonylperoxyacid	-0.114
ether (alicyclic)	-0.045
C=C (non-aromatic)	-0.043
peroxide	-0.027
nitro	-0.016
C=C-C=O in non-aromatic ring	-0.014
aromatic hydroxyl	-0.007
decane	-0.004
apin	-0.001
toluene	0.019
NumOfConfUsed	0.043
nitroester	0.051
aldehyde	0.052
nitrate	0.052
ester	0.069
ketone	0.165
NumOfN	0.183
carbonylperoxynitrate	0.224
log_pSat_Pa	1.000

Table 1: Correlation of features with `log_pSat_Pa`

2.2 Data preprocessing for model training

The training data was randomly split into 5 folds for cross-validation. By training and evaluating models on different subsets of the data, we can evaluate their

performance more reliably and reduce the risk of overfitting. Cross-validation provides a better estimate of how well a model will generalize to unseen data.

All features were scaled by removing the median and scaling them to the inter-quartile range. This method makes models more robust to outliers, as it relies on statistics that are less sensitive to extreme values. Scaling also ensures that all features are on a similar scale, preventing features with larger ranges from dominating the learning process.

3 Machine Learning

3.1 Considered approaches

The size of the training dataset offered itself up to not only use simple methods, but also fit more complex regressors, which might more accurately capture the nonlinear relationship between interpretable features and saturation pressure. We fitted parameters for and evaluated five different regressors, as well as a dummy predictor.

Dummy predictor
A dummy predictor predicts a constant value, in this case the mean of the target variable, for all inputs. While this obviously cannot capture relationships between features and the target variable, it serves as a baseline to evaluate the performance of more complex models: if their performance does not significantly exceed the dummy predictor's, it indicates a need to revisit feature engineering or model selection.

ElasticNet
ElasticNet is a linear regression model that combines L1 (Lasso) and L2 (Ridge) regularization. It balances feature selection (from L1) and coefficient shrinkage (from L2), making it effective when dealing with multicollinear or high-dimensional datasets. While ElasticNet models are fast to train and simple due to the L1 regularization removing irrelevant features, and might perform adequately if the target variable exhibits weak nonlinearity, ElasticNet is unlikely to match the accuracy of tree-based regressors for this regression problem, which we know to be highly nonlinear.

K-nearest neighbors
K-nearest neighbors (KNN) is a non-parametric regression method that predicts the target value for a given input based on the average of the target values of the k nearest training samples in feature space. The method is simple and intuitive, making no assumptions about the data distribution. However, KNN's performance heavily depends on the choice of k, the distance metric, and the scaling of features. It can be computationally expensive for large datasets since predictions require calculating distances to all training samples. Furthermore, KNN may struggle in high-dimensional spaces due to the "curse of dimensionality," which can dilute the significance of proximity.

Gradient-boosted trees
Gradient-boosted trees are an ensemble learning method that builds decision trees sequentially, where each new tree corrects the errors of the previous ones by minimizing a loss function. They are particularly effective for capturing complex relationships in nonlinear data and easily interpretable by looking at feature importance, but can be slow to train and are prone to overfitting.

Random forest
Random forest is an ensemble method that builds multiple decision trees independently and averages their predictions. It introduces randomness by selecting subsets of features and data samples for each tree. It is robust and generalizable, especially on larger datasets, and easily interpretable through feature importance analysis. Like gradient-boosted trees, they are more computationally expensive than other methods.

Support vector regression
Support vector regression (SVR) is a kernel-based method that aims to find a function that deviates from the actual target values by no more than a specified margin, while simultaneously minimizing model complexity. SVR is effective in capturing nonlinear relationships when combined with kernel functions such as the radial basis function (RBF) kernel. It is a powerful method for datasets where the relationship between features and the target variable is complex but smooth. It can, however, be computationally intensive, especially for large datasets, as it scales poorly with the number of samples.

3.2 Model Fitting & Feature Selection Process

We used Python with `sklearn 1.5.2`'s implementation of ElasticNet (`sklearn.linear_model.ElasticNet`), k-nearest neighbors (`sklearn.neighbors.KNeighborsRegressor`), random forest (`sklearn.ensemble.RandomForestRegressor`) and SVR (`sklearn.svm.SVR`). For gradient-boosted trees, `LGBMRegressor` from `lightgbm 4.5.0` was used.

Model parameters were first fit through 5-fold cross-validation using default hyperparameters to get an idea of the different methods' performance. In this step, to get a rough idea of feature importance, separate models were fit on four subsets of the training dataset: one containing all features, and three with all features with an absolute correlation of less than 5 % (10 features), 2.5 % (6 features) and 1% (3 features) (Table 1), respectively, removed.

In a second step, hyperparameters of each model were optimized using `Optuna` (`4.1.0`)'s tree-based Parzen estimator over 100 trials using the dataset with all features. The best combination of hyperparameters for each model was used to refit a new model on the entire training dataset afterwards.

Finally, feature importance was further analyzed by recursively eliminating features from suitable models (those assigning feature importance, i.e. random forest and gradient-boosted trees - ElasticNet had to be excluded due to computational infeasibility), and multiple different methods were combined using ensembling.

3.3 Models

3.3.1 ElasticNet

We trained ElasticNet with default parameters (equal contribution of L1 and L2 penalty, no multiplication of penalty terms) on polynomially expanded feature matrices of the training dataset. Polynomial degree 1 is equivalent to a linear regression, while a polynomial expansion of degree 2, for example, would add the square of each feature, as well as all products of two features. This expansion allows ElasticNet to capture more complex non-linear relationships, but is very computationally costly as the number of resulting features is polynomial in the number of base features and exponential in the degree. Empirically, computation of degrees over 4 was only realistically feasible for the dataset with 10 removed features.

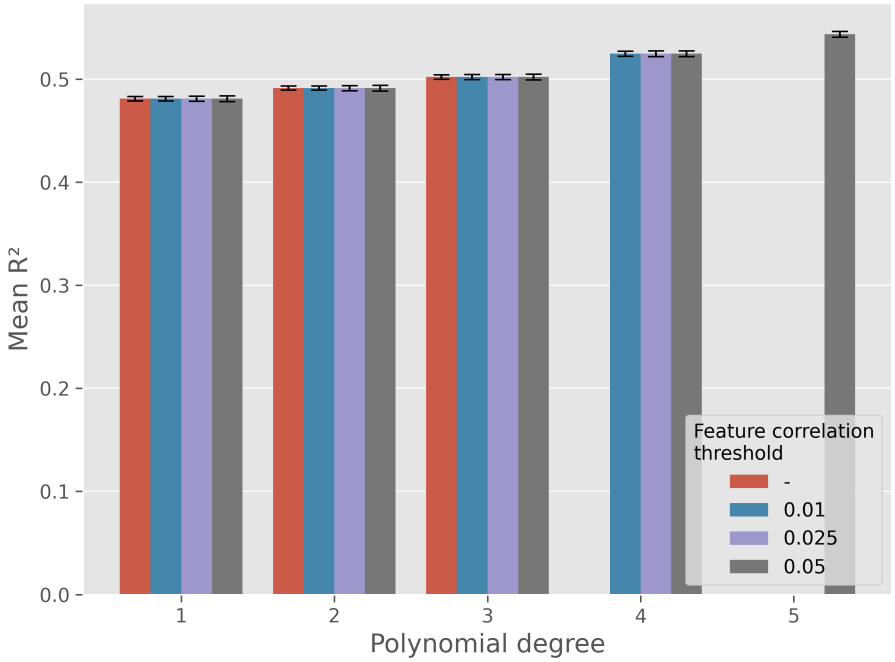


Figure 2: Overview of cross-validation R^2 for an ElasticNet polynomial degression depending on polynomial degree and correlation threshold for removed features. Error bars represent 95% symmetric confidence interval over cross-validation splits. Models at higher degrees only fit for fewer features due to computational constraints.

There was no difference in performance between different numbers of features included in the dataset (Figure 2), which is logical as ElasticNet's L1 regularization encourages feature selection. Model performance did increase significantly with each polynomial degree added (from 0.481±0.002 for degree 1 to 0.544±0.003 for degree 5), reflecting the high degree of nonlinearity of the relationship between covariates and logarithmic saturation vapour pressure.

Hyperparameter tuning (using degree 3 on the full dataset for computational reasons, Figure 3) led to a strongly improved model: at $\alpha = 0.07746$ and a penalty mixing ratio of 0.00018 L1 contribution, corresponding to a weakly penalized almost exclusively Ridge regressor (as the penalty is almost exclusively L2), a mean R^2 of 0.743±0.002 was achieved.

Refitting the model with these parameters on features that were polynomially expanded with degree 4 did not lead to a significant improvement (R^2 0.745±0.002).

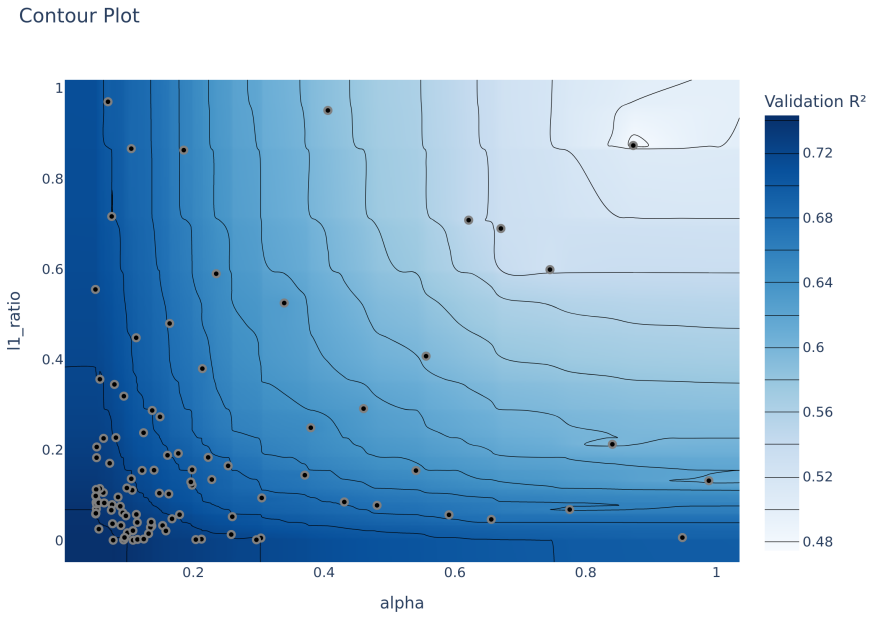


Figure 3: Contour plot of mean cross-validation R^2 for an ElasticNet polynomial regression (degree 3) depending on mixing parameter `l1_ratio` and penalty term multiplier `alpha`.

While recursive parameter elimination would have been interesting for this model, it was not possible in practice due to computational constraints.

3.3.2 K-nearest neighbors

Performance increased moderately with increasing number of neighbors (Figure 4). With only $k = 2$ neighbors, an R^2 score of 0.613±0.003 was reached. With higher numbers of neighbors, performance started plateauing from about $k = 14$ at 0.707±0.002 (higher values than $k = 19$ were not evaluated). Incorporating more neighbors into the prediction process allowed the model to better capture the underlying patterns in the data by reducing sensitivity to noise, until a plateau where averaging effects smoothes out useful local variation. This affirms KNN's suitability for the regression task given the highly complex nonlinear relationship of the multidimensional dataset, and suggests high relevance of local patterns in the feature space, which KNN can exploit. Additionally, KNN was the fastest method to fit in practice.

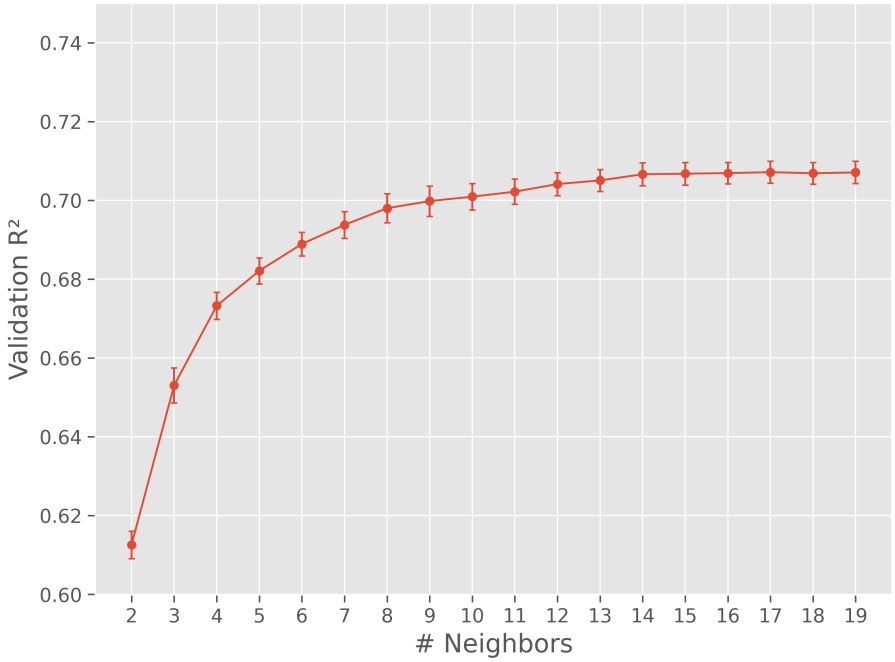


Figure 4: Cross-validation R^2 for k-nearest neighbors regressor using increasing number of neighbors. Error bars represent 95% symmetric confidence intervals computed from standard deviation of test score across cross-validation folds.

3.3.3 Gradient-boosted trees

Gradient-boosted trees with default hyperparameters (Table 2) achieved a mean R^2 of 0.744±0.003. Performance decreased with more dropped features, though this difference was only significant at a significance cutoff of 5% (i.e. 10 dropped features).

Hyperparameter tuning (Table 2) slightly increased the performance to 0.745±0.003, highlighting the robustness of this method to hyperparameters.

The most important features were structural ones (Figure 6), as suggested by the initial correlation analysis. Using 6 features already led to robust performance, and from 9 features on including additional features only led to marginal, performance improvements. Interestingly, after plateauing, performance made a small significant jump from including all features.

Hyperparameter	Default value	Tuned value
num_leaves	31	542
learning_rate	0.1	0.01
n_estimators	100	788
subsample	1.0	0.33
colsample_bytree	1.0	0.68
min_child_samples	20	30

Table 2: Default and optimized hyperparameters for gradient-boosted tree regression.

3.3.4 Random forest

The performance of random forests followed the same pattern as that of gradient-boosted trees (Figure 5), while being modestly lower: on all features, R^2 was 0.714±0.003, only significantly dropping when excluding the 10 features with correlation of less than 5 % with saturation vapour pressure to 0.686±0.003.

The best tuned random forest model (Table 3) was significantly better than the one with default parameters at an R^2 of 734±0.003, but still worse than even a non-tuned gradient-boosted tree regression.

Hyperparameter	Default value	Tuned value
n_estimators	100	498
min_samples_split	2	15
min_samples_leaf	1	1
max_features	1.0	0.3409

Table 3: Default and optimized hyperparameters for random forest regression.

By averaging predictions from multiple decision trees, random forests can effectively reduce variance and handle complex relationships in the data. However, they may perform slightly worse than gradient-boosted trees because they lack the iterative error correction that boosting provides, making them less capable of fine-tuning predictions in highly nonlinear problems.

Feature importance (Figure 7) generally mirrored that of gradient-boosted trees, with structural features being the most important ones, though notably molecular weight was much less important, while number of hydrogen bond donors was assigned much more importance.

Random forest regression also converged much faster than gradient-boosted trees, with using only one feature (number of hydrogen bond donors, which has the strongest correlation with logarithmic saturation vapour pressure) already

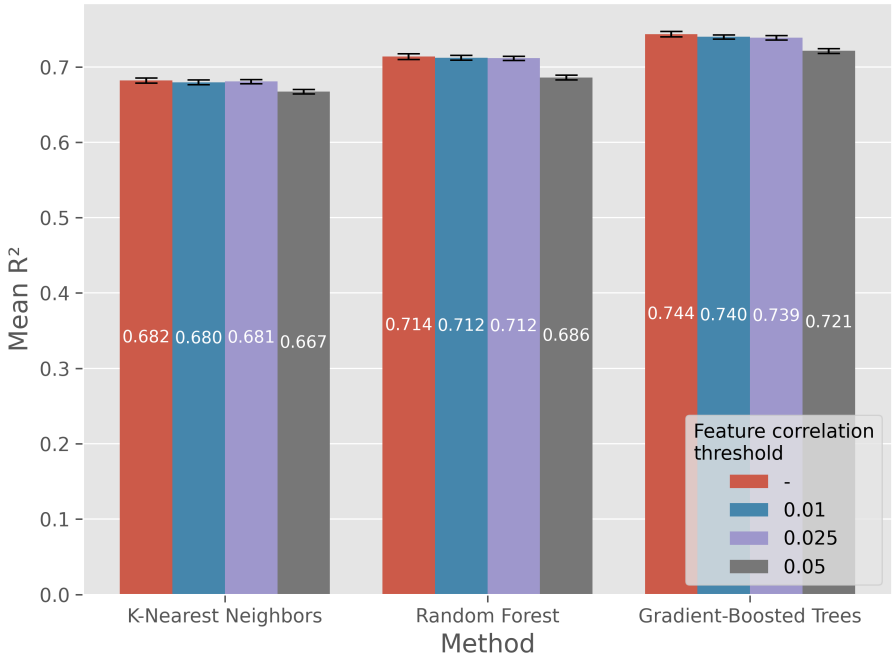


Figure 5: Cross-validation R^2 for k-nearest neighbors ($k = 5$), random forest, and gradient-boosted tree regressor on different subsets of data, using default hyperparameters. Error bars represent 95% symmetric confidence intervals computed from standard deviation of test score across cross-validation folds.

yielding an R^2 of 0.475±0.005.

Because the data contains features with strong individual predictive power, such as the number of hydrogen bond donors or number of stable conformers, which correlate significantly with saturation vapor pressure, the ability of random forests to select and heavily weight the most predictive feature early in the training process allows them to achieve robust performance with very simple imputs.

3.3.5 Support vector regression

Using a radial basis function kernel with default parameters ($C = 1$, $\varepsilon = 0.1$) led to the best results (R^2 0.746±0.003) among all default models, with a slight drop in performance from including fewer features (Figure 8).

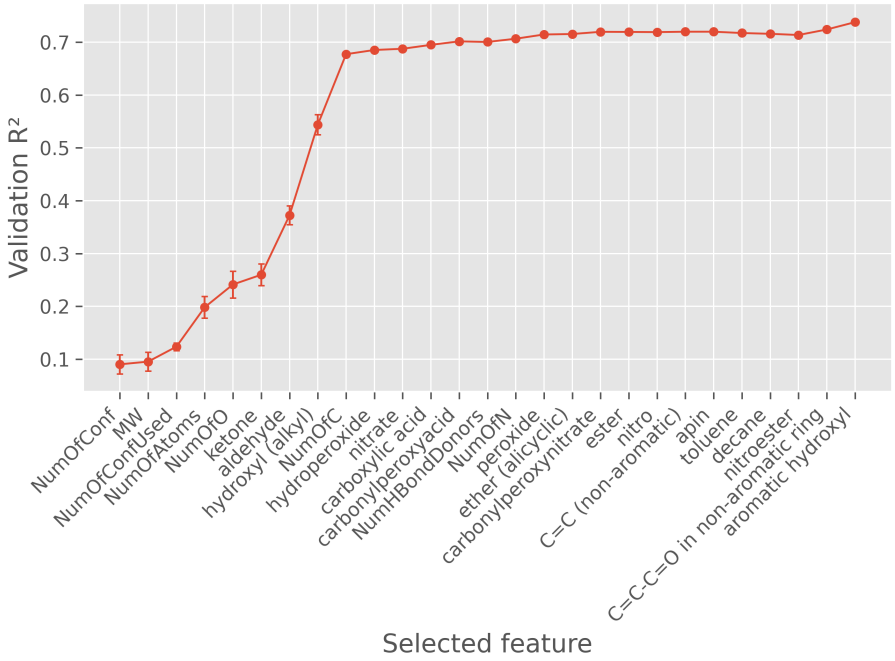


Figure 6: Cross-validation R^2 for gradient-boosted tree regressor fit on datasets with recursively eliminated features.

With a polynomial (degree 3) kernel, performance was lower (0.729±0.002) and the drop in performance from excluding features more significant. A linear kernel led to the worst performance (R^2 0.705±0.003), about the same as KNN regression.

The best resulting set of hyperparameters from hyperparameter tuning were a polynomial kernel of degree 3 (higher was not possible due to computational constraints), $C = 0.998$, $\varepsilon = 0.420$ and independent term of $c_0 = 0.979$, achieving a performance of 0.754±0.002, the highest of any model evaluated, through increased bias adjustment (increased c_0) and noise tolerance (increased ε).

Like ElasticNet, SVRs were very computationally expensive to train compared to the other methods.

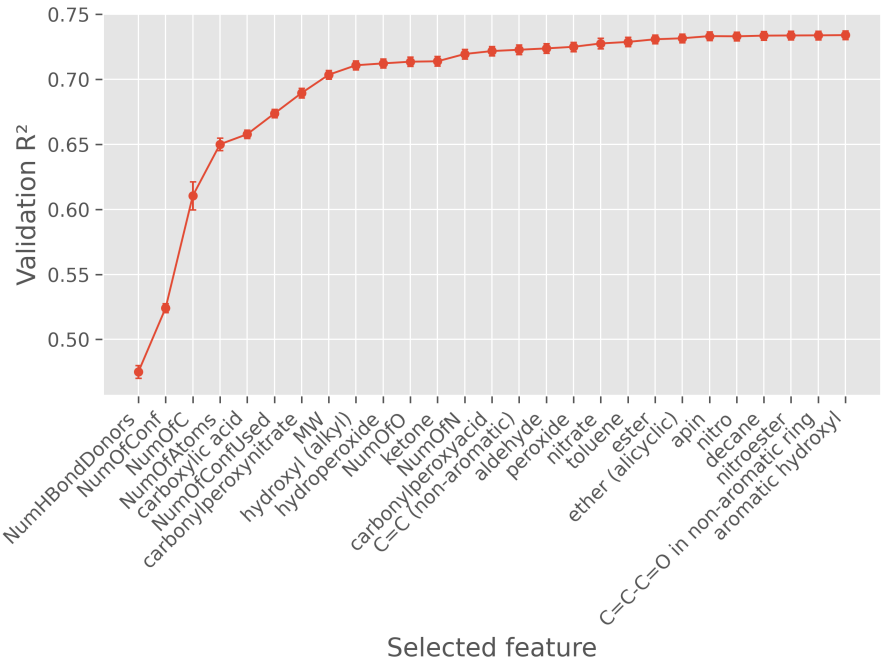


Figure 7: Cross-validation R^2 for random forest regressor fit on datasets with recursively eliminated features. Error bars represent 95% symmetric confidence intervals computed from standard deviation of test score across cross-validation folds.

3.4 Ensembling

To reduce bias and variance and combine strengths of different models, we first tried to train a stacking regressor from a KNN regressor, gradient-boosted tree regressor, and SVR, each with the optimal hyperparameter set from tuning and fit on the whole training dataset with all features. The stacking regressor trains these models on the data, then uses their predictions as inputs for a final meta-model, improving overall performance by leveraging the strengths of each individual model. The meta-model was a ridge regressor with its parameters tuned through 5-fold cross validation. This did not lead to a gain in performance over just the SVR model, likely because the model performances are already very similar and make similar errors, nullifying the advantage of reduced bias and variance gained from ensembling multiple high-bias weak learners.

For a weighted average ensemble of multiple models, we optimized model weights on the full training dataset, though this led to overfitting, giving almost exclusive weight (97%) to the gradient-boosted tree regressor. For explainabil-

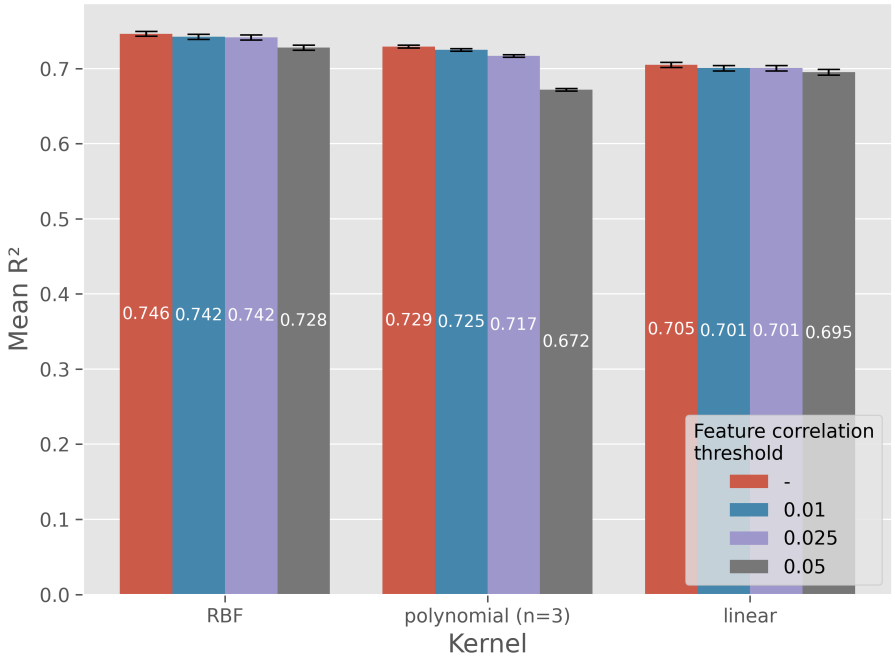


Figure 8: Cross-validation R^2 for support vector regression with different kernels fit on datasets with recursively eliminated features. Error bars represent 95% symmetric confidence intervals computed from standard deviation of test score across cross-validation folds.

ity's sake, we finally settled on an equally weighted mean of gradient-boosted trees and SVR, which is also the model whose test score we report.

4 Results & Discussion

While the dummy predictor obviously did not capture any relationship ($R^2 = 0$) between the features and outcome, all other evaluated predictors achieved robust performance over 5-fold cross validation. Not surprisingly, linear regression regularized with ElasticNet performed the worst of all (0.481 ± 0.002), although a parameter-tuned regression with polynomial degree 3 and almost exclusively L2 regularization achieved almost the same performance as other methods (0.743 ± 0.002).

Even though the relationship between covariates and logarithmic saturation

vapour pressure is complex and highly non-linear, ensembling methods (random forest and gradient-boosted trees), while less sensitive to hyperparameters, were not able to capture any more complex nonlinear relationships and did not result in a significant performance improvement over ElasticNet(and, as we know from the final presentations, the best-performing method from all groups was also a conceptually simple polynomial regression).

Optimized SVR did outperform the other methods slightly (R^2 of 0.754±0.002), while KNN regression underperformed relatively, while still achieving robust performance. ElasticNet and SVR took an order of magnitude more time to train than the tree-based methods, while KNN, which performed the worst among all methods, was by far the fastest.

Analyzing feature impotance revealed that a few features (mostly those containing structural information) carried a disproportionate amount of information, while molecular property-based features were much less important for predictive performance.

A very slight increase in predictive performance could be achieved by averaging the predictions of our two best-performing models, SVR and gradient-boosted trees, resulting in a public R^2 of 0.7620 and a private one of 0.7493, good for third on the respective leaderboards if submitted before the initial report deadline.

The multitude of well-performing models on the dataset not only highlights its high complexity, but also suggests it may have reached the performance ceiling for its current feature set, at least for somewhat simple statistical learning methods. Additional features, better preprocessing, or domain-specific insights might be needed to achieve further improvements.

5 Self-grading report

5.1 Report

We would grade our report with a 5. Our project demonstrates a thorough understanding of the topics covered in the lecture, applying machine learning techniques effectively to the GeckoQ dataset. The final report is polished, clearly structured, and adheres to all instructions provided.

The chosen machine learning models were appropriate for predicting saturation vapour pressure, and their implementation was well-justified and optimized.

The methodology has been analyzed in depth, with careful attention to feature selection and evaluation of model performance. Potential limitations were acknowledged and suitable tools (hyperparameter tuning, ensemble averaging) applied to increase model predictive power and robustness.

Our findings have been reported clearly. The project proceeded as planned and met all milestones within the given schedule.

5.2 Group

The group worked collaboratively and maintained an organized approach throughout the project. Tasks were distributed fairly, and all members actively contributed to discussions, analysis, and presentation. The group demonstrated strong cohesion, resulting in a project that significantly enhanced each member's understanding of the topic.

6 Appendix



Figure 9: Distribution of all variables in dataset.